

Programación por componentes

Semestre: 2026-1

Parcial 02

Profesor: Jorge Eduardo Hernández R.

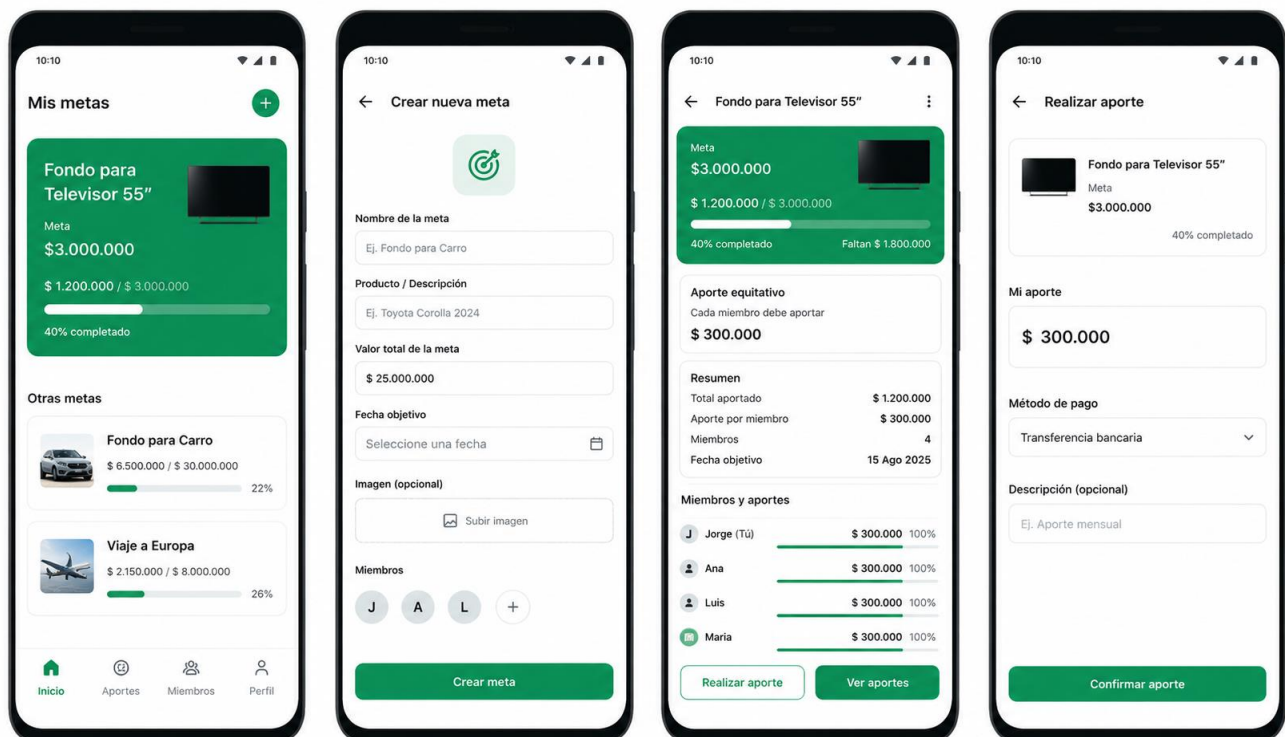
Instrucciones:

- Fecha de publicación: 11 de mayo de 2026
- Fecha de entrega: 27 de mayo de 2026.
- Medio de entrega: Sustentación en clase.
- La actividad debe ser en grupo.
- Se debe crear un repositorio git público.

Ejercicio

Una empresa necesita que estudiantes de tecnología en sistematización de datos desarrolle una aplicación para que las familias puedan realizar un ahorro para comprar un producto, ya sea un televisor, una moto, etc. La idea es poder agregar miembros al ahorro y que se pueda registrar los pagos de cada miembro.

Mockup



Notas:

- Es obligatorio que el proyecto implemente la arquitectura MVVM.
- Usar un backend propio es obligatorio. Lo recomendado es en NodeJS. No se puede usar firebase.
- No es necesario que haya login.
- La interfaz gráfica puede cambiar según como considere.
- Se debe usar Kotlin para desarrollar la aplicación
- Puede usar Jetpack Compose y/o XML para las pantallas gráficas.

- Uso exagerado de IA, en este caso más del 80% en el código, dará como resultado 0 en la nota final.
- Cualquier intento de copia o parecido con otros grupos se penalizará con cero para ambos grupos.

Calificación

Criterio	Descripción	Puntaje Máximo
1. Consumo de Servicios Web REST	Uso correcto de Retrofit para consumir el backend del sistema	2.0
Mostrar lista de metas desde API	Usa Retrofit para obtener las metas y mostrarlos dinámicamente en la UI	0.5
Ver detalle de una meta	Muestra información completa de la meta seleccionada miembros, aportes de cada miembro, resumen, valor total de la meta, foto de la meta, porcentaje faltante de la meta.	0.5
Registrar un pago (POST)**	Guarda un pago asociado a un miembro, ya sea del mismo o de otro integrante usando Retrofit	0.5
Consultar pagos de una meta	Obtiene del backend todos los pagos asociados a la meta	0.5
2. Uso del patrón MVVM	Implementación correcta de arquitectura y separación de responsabilidades	1.5
Lógica fuera de la UI (ViewModel limpio)	Activity/Fragment/Composable solo observa estados, nada de lógica dentro	0.5
LiveData o StateFlow para la UI	Manejo reactivo del estado y datos del usuario	0.5
Uso de Repository (API)	Repository como intermediario entre Retrofit y el ViewModel	0.5
3. Pruebas unitarias y validaciones	Validación del funcionamiento y reglas del negocio	0.5
Al menos una prueba sobre ViewModel o lógica	Ejemplo: validar cálculo del progreso de la meta, o mock de API	0.5
4. Código limpio y justificación técnica	Calidad del código y explicación del diseño	1.0
Código modular, limpio y legible	Convenciones, comentarios útiles, buenas prácticas	0.5
Justificación técnica del diseño	Explica por qué usó MVVM, Retrofit, Repository, etc.	0.5